Practica #3

Cristian miguel jimenez
Mecatrónica
ITLA
La Caleta, Santo Domingo
20240712@itla.edu.do

Resumen: Durante esta práctica, se estarán realizando una serie de ejercicios sobre multiplexores, circuitos combinacionales, y circuitos secuenciales, haciendo uso de todo lo aprendido a lo largo de la materia.

Abstract: During this practice, you will perform a series of exercises on multiplexers, combinational circuits, and sequential circuits, using everything you have learned throughout the course.

Palabras claves: lógica combinacional, lógica secuencial, multiplexores.

I. Introducción

Durante el siguiente documento, se estarán realizando varios ejercicios haciendo uso de todo lo aprendido en toda la materia, desde los circuitos combinacionales, hasta circuitos secuenciales usando FLIP-FLOP.

II. Marco Teórico

Circuitos lógicos: Es un conjunto de compuertas lógicas conectadas que procesan señales binarias (0 y 1) para realizar operaciones como suma, comparación o selección de datos, sin necesidad de memoria.

Circuito combinacional: Es un tipo de circuito lógico cuya salida depende únicamente del estado actual de sus entradas, sin considerar entradas anteriores o almacenamiento de datos.

Registros: Un registro es un conjunto de flip-flops conectados entre sí que sirve para almacenar y manipular varios bits de información de manera simultánea.

Circuito secuencial: Un circuito secuencial es un tipo de circuito digital cuya salida depende no solo de las entradas actuales, sino también del historial de entradas anteriores. Esto se logra mediante elementos de memoria como flip-flops, permitiendo que el circuito "recuerde" estados pasados y tenga comportamientos temporales, como contar, almacenar datos o seguir secuencias.

Multiplexor: Un multiplexor (MUX) es un dispositivo digital que selecciona una de varias señales de entrada y la dirige a una sola salida, según el valor de unas señales de control. Es como un “interruptor electrónico” que permite controlar qué entrada se transmite, optimizando el uso de canales o recursos.

Flip-flop: Un flip-flop es un dispositivo electrónico básico de memoria que puede almacenar un solo bit de información. Cambia su estado de salida en respuesta a señales de control, como relojes o entradas lógicas, y es fundamental en circuitos secuenciales y sistemas digitales como contadores y registros.

Verilog: Verilog es un tipo de HDL muy popular que se usa para modelar sistemas digitales; permite escribir código que describe cómo debe funcionar un circuito y luego sintetizarlo para implementarlo físicamente.

SOP (Suma de Productos): es una expresión lógica donde se suman (OR) términos que son productos (AND) de variables o sus negaciones. Ejemplo: A'B + AB'.

POS (Producto de Sumas): es una expresión donde se multiplican (AND) términos que son sumas (OR) de variables o sus negaciones. Ejemplo: (A + B') (A' + B).

III. Ejercicio sobre MUX #1

"A. Se requiere implementar la siguiente función usando un mux 2x1. Donde:

Fw (A, B, C, D) = (1,2,4,5,6,11,12,15).

Tome en consideración que se usará producto de suma de dicha función.

b. Se requiere conectar un mux 4:1 donde la función es:

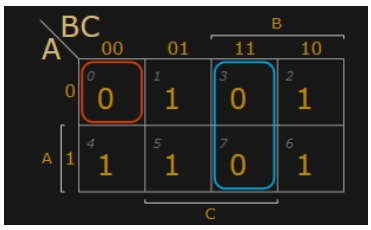
F (A, B, C, D, E) = (0, 1, 3, 5, 8, 9, 11, 14, 15, 17, 20, 21, 22, 23) + d (25, 27, 30)"

Primero, haciendo uso de VEM, realizamos la tabla de verdad del ejercicio a:

Tabla de verdad				
A	B	C	D	F
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

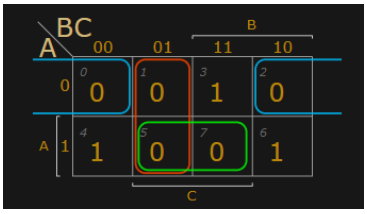
Tabla de verdad

Partiendo de la tabla de verdad, desarrollamos los Kmaps usando la calculadora y obtenemos las funciones:



Primer kmap

$$fa = (B' + C')(A + B + C)$$



Segundo K-map

$$fb = (A + C)(B + C')(A' + C')$$

Ahora partiendo de esas funciones es creamos los diagramas en multism:

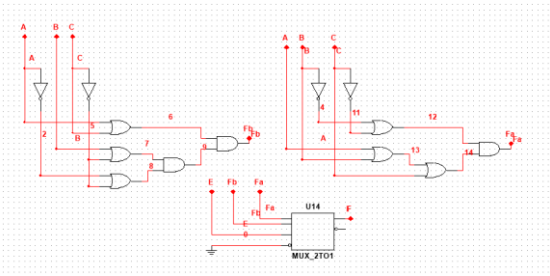


Diagrama lógico

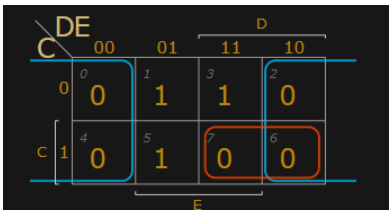
Ahora, podemos crear la tabla de verdad del apartado B.

Nota: Los números 25, 27, 30 los tomaremos como don't care.

Tabla de verdad					
A	B	C	D	E	F
0	0	0	0	0	0
0	0	0	0	1	1
0	0	0	1	0	0
0	0	0	1	1	1
0	0	1	0	0	0
0	0	1	0	1	1
0	0	1	1	0	0
0	0	1	1	1	0
0	1	0	0	0	1
0	1	0	0	1	1
0	1	0	1	0	0
0	1	0	1	1	1
0	1	1	0	0	0
0	1	1	0	1	0
0	1	1	1	0	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	0	1	1
1	0	0	1	0	0
1	0	0	1	1	0
1	0	1	0	0	1
1	0	1	0	1	1
1	0	1	1	0	1
1	0	1	1	1	1
1	1	0	0	0	0
1	1	0	0	1	X
1	1	0	1	0	0
1	1	0	1	1	X
1	1	1	0	0	0
1	1	1	0	1	0
1	1	1	1	0	X
1	1	1	1	1	0

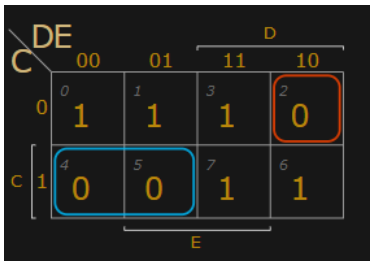
Tabla de verdad

Ahora creamos los kmaps:



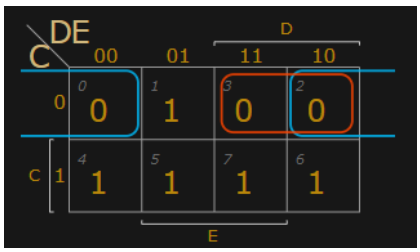
Primer kmap

$$fa = (E)(C' + D')$$



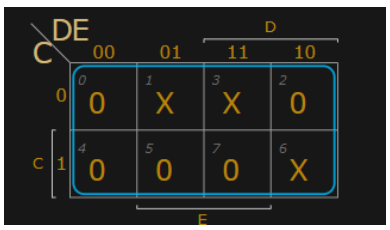
Segundo Kmap

$$fb = (C' + D)(C + D' + E)$$



Tercer Kmap

$$fc = (C + E)(C + D')$$



Cuarto kmap

$$Fd = 0$$

Ahora realizamos el diagrama:

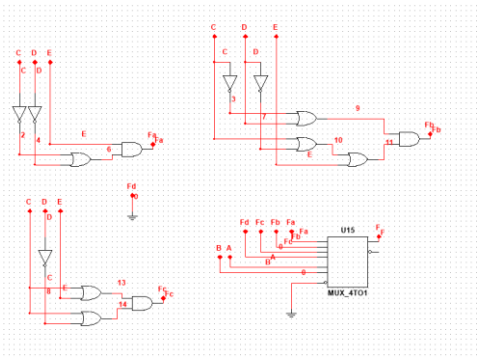


Diagrama de segundo ejercicio mux

IV. Ejercicio #2

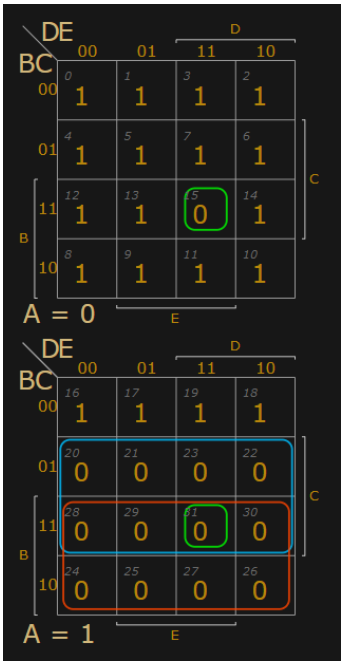
"Una nave industrial consta de dos portones de entrada (P1, P2) y dos grandes ventanales (V1, V2). Un interruptor accionado por llave (I), indica si se está en jornada laboral o fuera de ella (nave cerrada). Durante la jornada laboral se permite que esté abierto uno de los dos portones, pero no los dos al mismo tiempo, pudiendo estar cualquier ventana abierta. Fuera de la jornada laboral, no podrán estar, ni los portones ni las ventanas, abiertas. Diseña un sistema de alarma que detecte estas situaciones. Consideramos que puerta o ventanas abiertas es 0 e interruptor conectado es 1. Resuelva por producto de sumas."

El primer paso de todos es crear la tabla de verdad:

Tabla de verdad					
I (A)	P1 (B)	P2 (C)	V1 (D)	V2 (E)	SA
0	0	0	0	0	1
0	0	0	0	1	1
0	0	0	1	0	1
0	0	0	1	1	1
0	0	1	0	0	1
0	0	1	0	1	1
0	0	1	1	0	1
0	0	1	1	1	1
0	1	0	0	0	1
0	1	0	0	1	1
0	1	0	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	0	0	1	1
1	0	0	1	0	1
1	0	0	1	1	1
1	0	1	0	0	0
1	0	1	0	1	0
1	0	1	1	0	0
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	0	1	0
1	1	0	1	0	0
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	0	1	0
1	1	1	1	0	0
1	1	1	1	1	0

Tabla de verdad

Al tener 5 entradas, tendremos dos kmaps en el desarrollo:



Kmaps

Y la función resultante es:

$$Sa = (A' + C')(A' + B')(B' + C' + D' + E')$$

Usando esa función, crearemos el diagrama lógico y el código haciendo uso de verilog.

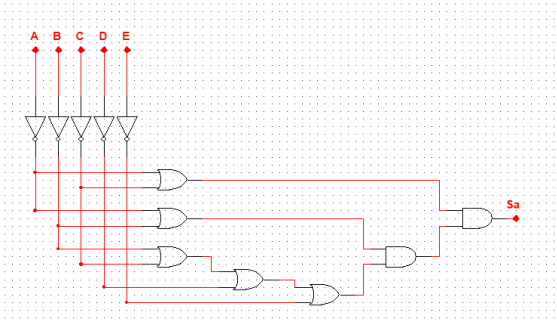


Diagrama en multisim

```
module ejercicio_2(
    input A, B, C, D, E,
    output Sa
);

    assign Sa = ( (~A) | (~C) ) & ( (~A) | (~B) ) & ( (~B) | (~C) | (~D) | (~E) );

endmodule
```

Función en verilog

V. Ejercicio #3

"El ingeniero de turno ha estado trabajando en un diseño de un FSM (Finite State Machine) y no ha tenido tiempo para completarlo, así que sin dar muchas explicaciones ha dejado un diagrama de estado y nos ha pedido que realicemos un circuito secuencial con las siguientes características. usando flip flops RS. Además de esto ha dejado una nota que dice use productos de suma para el diseño. Las variables serán Io, I1 e I2 donde Io es la menos significativa."

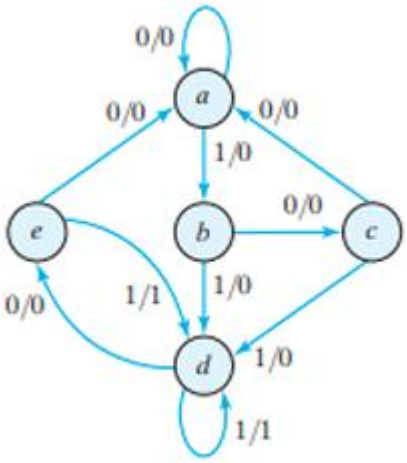


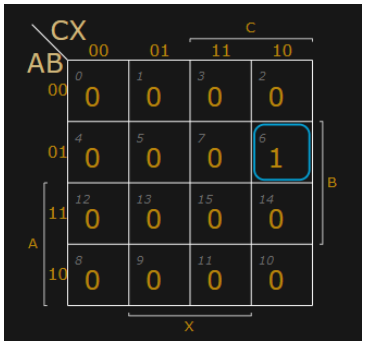
Fig 1: Ejercicio 2

El primer paso será construir nuestra tabla de estados y a partir de ahí desarrollar nuestro circuito secuencial:

Tabla de verdad												
Estado presente			In	Estado futuro			F-F Tipo RS					
I2(A)	I1(B)	I0(C)	X	FI2	FI1	FI0	SI2	RI2	SI1	RI1	SI0	RI0
0	0	0	0	0	0	0	0	X	0	X	0	X
0	0	0	1	0	0	1	0	X	0	X	1	0
0	0	1	0	0	1	0	0	X	1	0	0	1
0	0	1	1	0	1	1	0	X	1	0	X	0
0	1	0	0	0	0	0	0	X	0	1	0	X
0	1	0	1	0	1	1	0	X	X	0	1	0
0	1	1	0	1	0	0	1	0	0	1	0	1
0	1	1	1	0	1	1	0	X	X	0	X	0
1	0	0	0	0	0	0	0	1	0	X	0	X
1	0	0	1	0	1	1	0	1	1	0	1	0

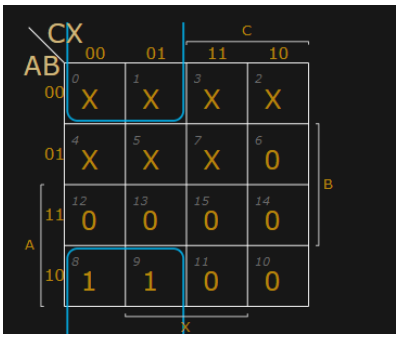
Tabla de estados

A partir de las tablas de estados, buscamos los circuitos combinacionales de los flip-flop.



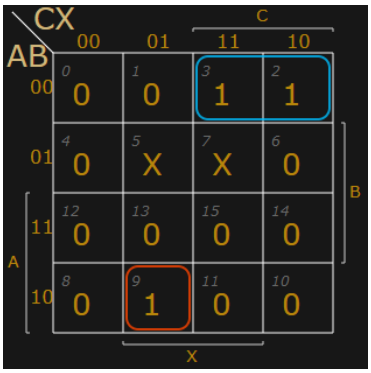
SI2

$$Si2 = A'BCX'$$



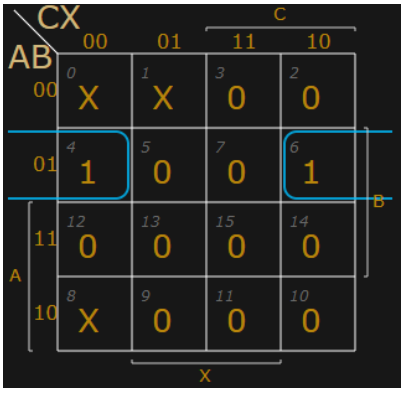
RI2

$$RI2 = B'C'$$



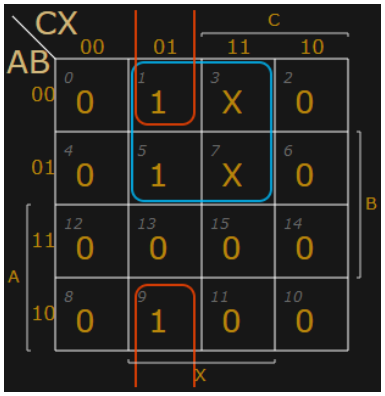
SI1

SI1 = A'B'C + AB'C'X



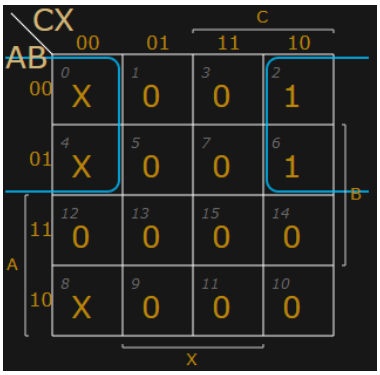
RI1

RI1 = A'BX'



SI0

SI0 = A'X + B'C'X



RI0

RI0 = A'X'

Ahora, creamos el diagrama y el código para la implementación en verilog:

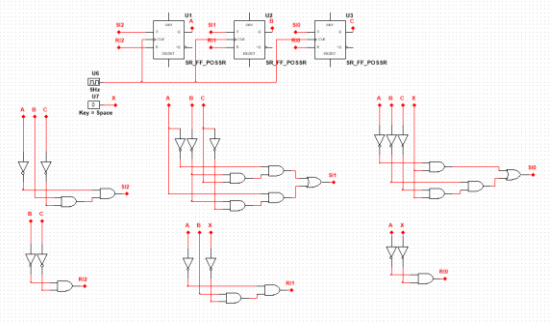


Diagrama en multisim

```
module Ejercicio3(
    input A, B, C, X, clk,
    output Q2, Q1, Q0
);

wire S2, R2, S1, R1, S0, R0;

assign S2 = (~A & B & C & ~X);
assign R2 = (~B & ~C);

assign S1 = (~A & ~B & C) | (A & ~B & ~C & X);
assign R1 = (~A & B & ~X);

assign S0 = (~A & X) | (~B & ~C & X);
assign R0 = (~A & ~X);

RS_ff FF2 (S2, R2, clk, Q2);
RS_ff FF1 (S1, R1, clk, Q1);
RS_ff FF0 (S0, R0, clk, Q0);

endmodule
```

Código en verilog

VI. Ejercicio #4

"De los siguientes diagramas lógicos deduzca la tabla de verdad y escriba la función booleana resultante."

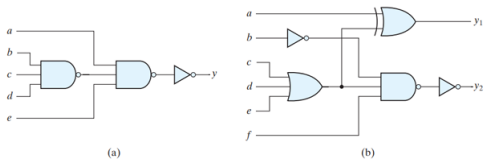


Fig 1: Ejercicio 3

Las funciones de los anteriores circuitos son:

A.) $y = (((BCD)'AE)')'$

B.) $y1 = (C + D + E)B'F$

$$y2 = A \oplus (C + D + E)$$

Y las tablas de verdad son:

Tabla de verdad					
A	B	C	D	E	Y
0	0	0	0	0	1
0	0	0	0	1	1
0	0	0	1	0	1
0	0	0	1	1	1
0	0	1	0	0	1
0	0	1	0	1	1
0	0	1	1	0	1
0	0	1	1	1	1
0	1	0	0	0	1
0	1	0	0	1	1
0	1	0	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	1
1	0	0	0	0	1
1	0	0	0	1	1
1	0	0	1	0	1
1	0	0	1	1	1
1	0	1	0	0	1
1	0	1	0	1	1
1	0	1	1	0	1
1	0	1	1	1	1
1	1	0	0	0	1
1	1	0	0	1	1
1	1	0	1	0	1
1	1	0	1	1	1
1	1	1	0	0	1
1	1	1	0	1	1
1	1	1	1	0	1
1	1	1	1	1	0

Tabla de verdad, primera función

Tabla de verdad							
A	B	C	D	E	F	Y1	Y2
0	0	0	0	0	0	0	0
0	0	0	0	0	1	0	0
0	0	0	0	1	0	1	0
0	0	0	0	1	1	1	1
0	0	0	1	0	0	1	0
0	0	0	1	0	1	1	1
0	0	0	1	1	0	1	0
0	0	0	1	1	1	1	1
0	0	1	0	0	0	1	0
0	0	1	0	0	1	1	1
0	0	1	0	1	0	1	0
0	0	1	0	1	1	1	1
0	0	1	1	0	0	1	0
0	0	1	1	0	1	1	1
0	0	1	1	1	0	1	0
0	0	1	1	1	1	1	1
0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0
0	1	0	0	1	0	1	0
0	1	0	0	1	1	1	0
0	1	0	1	0	0	1	0
0	1	0	1	0	1	1	0
0	1	0	1	1	0	1	0
0	1	0	1	1	1	1	0
0	1	1	0	0	0	1	0
0	1	1	0	0	1	1	0
0	1	1	0	1	0	1	0
0	1	1	0	1	1	1	0
0	1	1	1	0	0	1	0
0	1	1	1	0	1	1	0
0	1	1	1	1	0	1	0
0	1	1	1	1	1	1	0
1	0	0	0	0	0	1	0
1	0	0	0	0	1	1	0
1	0	0	0	1	0	0	0
1	0	0	0	1	1	0	1
1	0	0	1	0	0	0	0
1	0	0	1	0	1	0	1
1	0	0	1	1	0	0	0
1	0	0	1	1	1	0	1
1	0	1	0	0	0	0	0
1	0	1	0	0	1	0	1
1	0	1	0	1	0	0	0
1	0	1	0	1	1	0	1
1	0	1	1	0	0	0	0
1	0	1	1	0	1	0	1
1	0	1	1	1	0	0	0
1	0	1	1	1	1	0	0
1	1	0	0	0	0	1	0
1	1	0	0	0	1	1	0
1	1	0	0	1	0	0	0
1	1	0	0	1	1	0	0
1	1	0	1	0	0	0	0
1	1	0	1	0	1	0	0
1	1	0	1	1	0	0	0
1	1	0	1	1	1	0	0
1	1	1	0	0	0	0	0
1	1	1	0	0	1	0	0
1	1	1	0	1	0	0	0
1	1	1	0	1	1	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	1	0	0
1	1	1	1	1	0	0	0
1	1	1	1	1	1	0	0

A partir de esta información, implementamos haciendo uso de verilog:

```
module Ejercicio4_Y(
    input A, B, C, D, E,
    output y
);

assign y = ~(~(~(B & C & D) | (A & E)));

endmodule
```

Primer diagrama

```
module Ejercicio4_Y1Y2(
    input A, B, C, D, E, F,
    output y1, y2
);

assign y1 = ((C | D | E) & (~B & F));
assign y2 = A ^ (C | D | E);

endmodule
```

Segundo diagrama

VII. Ejercicio #5

"Del siguiente diagrama de tiempo deduzca la tabla de verdad y la función booleana complementada, siendo A, B y C los bits de entrada y Fs la función de salida. (tome los picos de las entradas y la salida como interferencias indeseadas)."

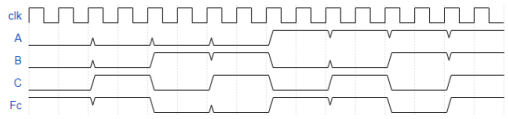


Fig 2: Ejercicio 5

A partir del este diagrama de tiempos, creamos nuestra tabla de verdad:

Tabla de verdad			
A	B	C	Fc
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

S0	00	01	11	10
0	1	1	0	0
1	1	1	1	0

$$Fc = B' + AC$$

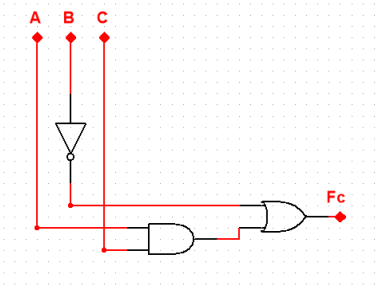


Diagrama lógico

```
module ejercicio5(
    input wire A, B, C,
    output wire Fc
);

assign Fc = ~B | A & C ;

endmodule
```

Código en verilog

VIII. Ejercicio #6

"Implementar un contador Jhonson y un anular de tres bits en un software de simulación."

Primero que nada, hay que saber que es un contador Jhonson y el contador anular:

Contador Jhonson: Un contador anular es un tipo de contador secuencial en el que un solo bit "1" (o "0") circula entre un registro de flip-flops conectados en cadena. Cada pulso de reloj desplaza el bit activo a la siguiente posición, y cuando llega al final, regresa al inicio, repitiendo el ciclo. Si el contador tiene n flip-flops, genera n estados distintos, lo que lo hace útil para secuencias cíclicas, control de LEDs o selección de operaciones de manera repetitiva.

Contador Anular: Un contador Johnson, también llamado contador en anillo invertido, es una variante del contador anular donde el complemento del último bit se retroalimenta a la entrada del primer flip-flop. Esto genera una secuencia de bits en la que los unos y ceros se expanden y contraen a través del registro, permitiendo que n flip-flops produzcan 2n estados distintos. Es muy utilizado en decodificadores, secuenciadores y máquinas de estados donde se requiere un mayor número de estados con la misma cantidad de flip-flops.

Sabiendo esto, los diagramas son:

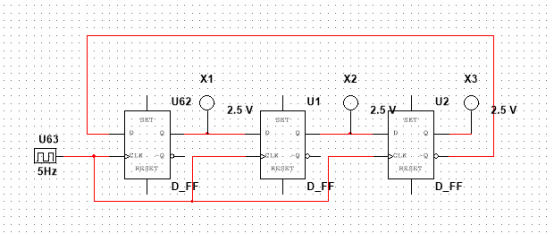


Diagrama contador Jhonson

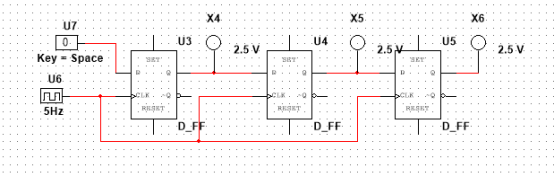
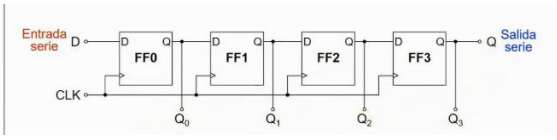


Diagrama contador Anular

IX. Ejercicio #7

"Explique el funcionamiento de un registro Universal, anexe diagrama y defina los diferentes tipos de registros que podemos encontrar."

Un registro universal es un tipo de circuito secuencial que se utiliza para guardar información digital y manipularla de diversas formas, dependiendo del modo de operación que se le indique. Este componente puede realizar múltiples funciones: cargar datos en paralelo, desplazar los bits hacia la izquierda o hacia la derecha, o conservar el valor almacenado sin alteraciones. Gracias a esta versatilidad, se convierte en una herramienta clave en arquitecturas digitales como procesadores, donde se requiere manejar datos de forma ordenada y temporal.



Registro de desplazamiento

Existen varios tipos de registros, cada uno con características específicas:

- **Registro de almacenamiento**: Su función principal es retener datos. La carga se realiza de manera paralela, y no permite desplazamientos.
- **Registro de desplazamiento**: Está diseñado para mover bits en serie, ya sea hacia la izquierda o hacia la derecha. Es útil en tareas como la conversión entre formatos seriales y paralelos o en la implementación de retardos.
- **Registro bidireccional**: Permite el desplazamiento de bits en ambas direcciones, lo que lo hace más flexible que el registro de desplazamiento simple.
- **Registro universal**: Reúne todas las funciones anteriores. Puede cargar datos en paralelo, desplazar bits en ambos sentidos y mantener su contenido sin cambios, según las señales de control que reciba. Por ello, se considera una unidad de almacenamiento temporal altamente adaptable.

En resumen, el registro universal actúa como una herramienta multifuncional dentro de los sistemas digitales, capaz de ajustarse a distintas operaciones según lo requiera el diseño del circuito.

X. Ejercicio #8

"Defina que es un ADC (Analog Digital converter) y un DAC anexe explicacion minimo 3 parrafo, anexe fotos y diagramas y 10 aplicaciones donde se utilizan estos conversores. Anexe info sobre un IC comercial de 12 bits que sea un ADC y otro de DAC."

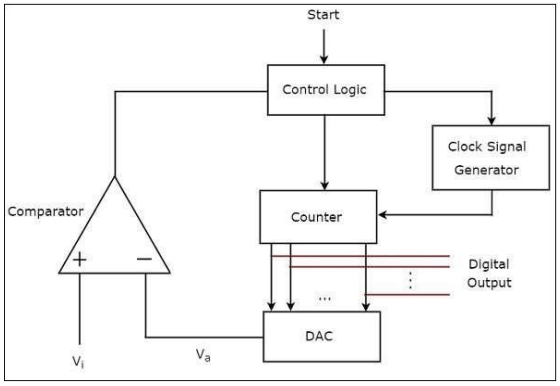
ADC (Analog to Digital Converter):

Un ADC, o conversor analógico-digital, es un dispositivo electrónico que convierte una señal analógica continua (como voltaje o corriente) en una señal digital discreta que pueda ser procesada por sistemas digitales, como microcontroladores o computadoras. La señal analógica puede tomar infinitos valores dentro de un rango, mientras que la salida digital tiene un número limitado de niveles determinados por el número de bits del ADC.

El funcionamiento básico de un ADC implica muestreo, donde se toma el valor de la señal en instantes específicos de tiempo, y cuantificación, donde el valor analógico se aproxima al nivel digital más cercano. Entre los tipos más comunes de ADC se encuentran flash, sucesiva aproximación, sigma-delta y rampas, cada uno con ventajas según la velocidad, resolución y complejidad requerida.

Un diagrama típico de un ADC incluye:

1. Entrada analógica.
2. Etapa de muestreo y retención (Sample & Hold).
3. Comparador o modulador según el tipo de ADC.
4. Lógica de conversión que entrega la salida digital.



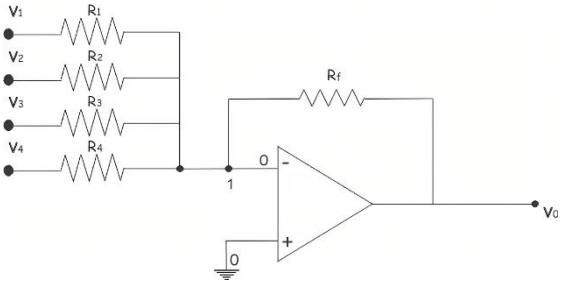
DAC (Digital to Analog Converter)

Un DAC, o conversor digital-analógico, realiza la operación inversa del ADC: convierte una señal digital en una señal analógica proporcional. La entrada es un número binario que representa el valor deseado, y la salida es un voltaje o corriente que puede variar de forma continua dentro de un rango.

El funcionamiento de un DAC suele basarse en redes de resistencias ponderadas, escalones de corriente o técnicas de sigma-delta, dependiendo de la resolución y la precisión requerida. Los DAC permiten, por ejemplo, generar señales de audio, controlar motores, producir formas de onda analógicas o calibrar instrumentos de medición desde un sistema digital.

Un diagrama típico de un DAC incluye:

1. Entrada digital (bits).
2. Red de resistencias o circuitos de conversión.
3. Salida analógica que puede ser corriente o voltaje.



Aplicaciones de ADC y DAC

- Audio digital: grabación y reproducción de sonido.
- Instrumentación médica: monitoreo de ECG, sensores de presión y temperatura.
- Telemetría: adquisición de datos desde sensores remotos.
- Control de motores: ajuste de velocidad y posición mediante señales analógicas.
- Electrónica de consumo: cámaras digitales, micrófonos, televisores.
- Automatización industrial: supervisión y control de procesos.
- Telecomunicaciones: modulación y demodulación de señales analógicas.
- Sistemas de radar y sonar: digitalización de señales recibidas.
- Medición en laboratorio: voltímetros y osciloscopios digitales.
- Instrumentos musicales electrónicos: sintetizadores y controladores MIDI.

ADC de 12 bits – MCP3201 (Microchip)

El MCP3201 es un convertidor analógico a digital de 12 bits con interfaz SPI, ideal para aplicaciones donde se requiere una adquisición de datos precisa y eficiente.

- Conversión: Analógica a digital (ADC).
- Rango de entrada: 0 a 5 V, compatible con sensores de voltaje estándar.
- Resolución: 12 bits, lo que permite representar hasta 4096 niveles discretos de voltaje.
- Interfaz de comunicación: SPI, lo que facilita su integración con microcontroladores y sistemas embebidos.
- **Aplicaciones típicas**:
    - Lectura de sensores analógicos (temperatura, presión, luz).
    - Sistemas de adquisición de datos.
    - Equipos de monitoreo ambiental.
    - Instrumentación portátil y médica.

Este ADC se destaca por su bajo consumo de energía y su capacidad de operar en tiempo real, lo que lo hace adecuado para sistemas que requieren respuesta rápida y precisión.

DAC de 12 bits – MCP4921 (Microchip)

El MCP4921 es un convertidor digital a analógico de 12 bits que permite generar señales analógicas a partir de datos digitales, también con interfaz SPI.

- Conversión: Digital a analógica (DAC).
- Rango de salida: 0 a 5 V, ajustable según la referencia de voltaje utilizada.
- Resolución: 12 bits, permitiendo una salida analógica con 4096 niveles de precisión.
- Interfaz de comunicación: SPI, compatible con microcontroladores y FPGAs.
- Aplicaciones típicas:
    - Generación de formas de onda (senoidal, triangular, etc.).
    - Control de actuadores (motores, válvulas, servos).
    - Calibración de equipos de medición.
    - Sistemas de audio y síntesis musical.

Este DAC es ideal para sistemas que requieren una salida analógica suave y controlada, como en control de procesos, instrumentación industrial o simulación de señales.

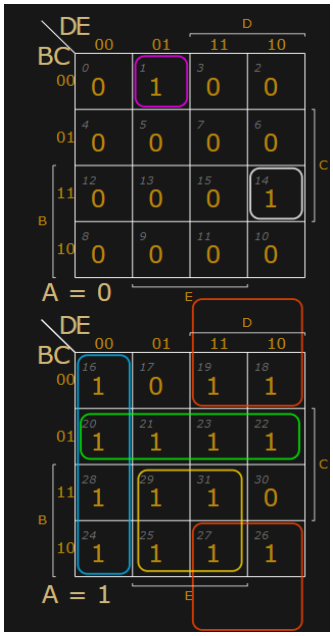
XI. Ejercicio #9

"Realizar un contador que de 4 bits que cuente ascendentemente y luego descienda hasta 4'b0000 en un bucle infinito. Use un flip flop tipo D"

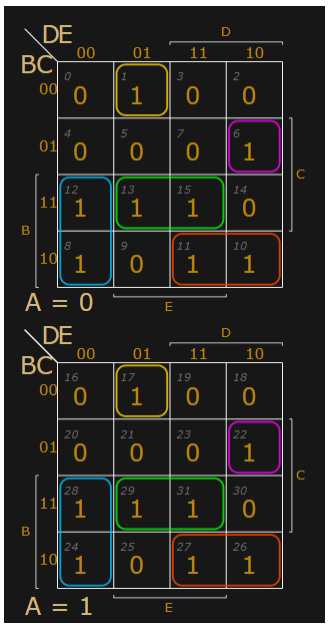
Primero desarrollaremos la tabla de estados:

Tabla de verdad												
Estado presente					Estado futuro				F-F Tipo D			
A1	B1	C1	D1	X	A2	B2	C2	D2	Da	Db	Dc	Dd
0	0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	1	1	1	1	1	1	1	1	1
0	0	0	1	0	0	0	1	0	0	0	1	0
0	0	0	1	1	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	1	1	0	0	1	1
0	0	1	0	1	0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0	0	0	1	0	0
0	0	1	1	1	0	0	1	0	0	0	1	0
0	1	0	0	0	0	1	0	1	0	1	0	1
0	1	0	0	1	0	0	1	1	0	0	1	1
0	1	0	1	0	0	1	1	0	0	1	1	0
0	1	0	1	1	0	1	0	0	0	1	0	0
0	1	1	0	0	0	1	1	1	0	1	1	1
0	1	1	0	1	0	1	0	1	0	1	0	1
0	1	1	1	0	1	0	0	0	1	0	0	0
0	1	1	1	1	0	1	1	0	0	1	1	0
1	0	0	0	0	1	0	0	1	1	0	0	1
1	0	0	0	1	0	1	1	1	0	1	1	1
1	0	0	1	0	1	0	1	0	1	0	1	0
1	0	0	1	1	1	0	0	0	1	0	0	0
1	0	1	0	0	1	0	1	1	1	0	1	1
1	0	1	0	1	1	0	0	1	1	0	0	1
1	0	1	1	0	1	1	0	0	1	1	0	0
1	0	1	1	1	1	0	1	0	1	0	1	0
1	1	0	0	0	1	1	0	1	1	1	0	1
1	1	0	0	1	1	0	1	1	1	0	1	1
1	1	0	1	0	1	1	1	0	1	1	1	0
1	1	0	1	1	1	1	0	0	1	1	0	0
1	1	1	0	0	1	1	1	1	1	1	1	1
1	1	1	0	1	1	1	0	1	1	1	1	1
1	1	1	1	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	0	1	1	1	0

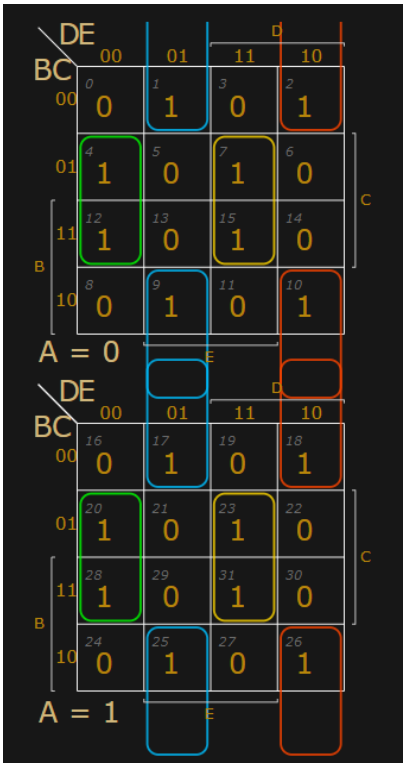
A partir de esta tabla de estados, creamos los kmaps:



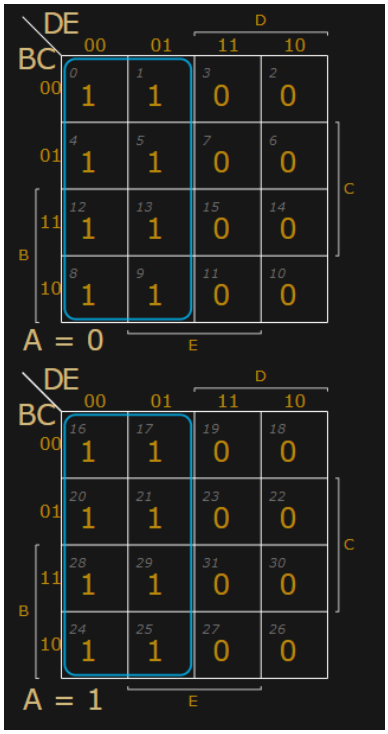
Da = AD'E' + AC'D + AB'C + ABE + A'B'C'D'E + A'BCDE'



Db = BD'E' + BC'D + BCE + B'C'D'E + B'CDE'



Dc = C'D'E + C'DE' + CD'E' + CDE



Dd = D'

Ya con las funciones, creamos el diagrama en multisim:

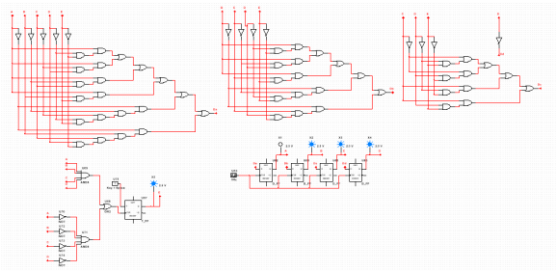


Diagrama lógico

Nota: Se agrego un Flip flop retensivo para poder invertir la secuencia al llegar un punto.

Y para finalizar, la implementación en verilog:

```
1  module ejercicio9(
2      input wire clk,
3      output wire A, B, C, D
4  );
5      wire clock ;
6          clk U_CLOCK(
7          .clock_in(clk),
8          .clock_out(clock)
9      ) ;
10     wire nA, nB, nC, nD, nE
11     wire Da, Db, Dc, Dd, E
12
13     //Uso de FF
14     ff_d U_da(
15         .clk(clock),
16         .rst_n(1'b1),
17         .D(Da),
18         .Q(A)
19     );
20     not U_NOTA(nA, A) ;
21
22     ff_d U_db(
23         .clk(clock),
24         .rst_n(1'b1),
25         .D(Db),
26         .Q(B)
27     );
28     not U_NOTB(nB, B) ;
29
30     ff_d U_dc(
31         .clk(clock),
32         .rst_n(1'b1),
33         .D(Dc),
34         .Q(C)
35     );
36     not U_NOTC(nC, C) ;
37
38     ff_d U_dd(
39         .clk(clock),
40         .rst_n(1'b1),
41         .D(Dd),
42         .Q(D)
43     );
44     not U_NOTD(nD, D) ;
```

```
48      //Funciones
49      wire a0, a1, a2, a3, a4, a5 ;
50      and U_AND00(a0, A, nD, nE);
51      and U_AND01(a1, A, nC, D);
52      and U_AND02(a2, A, nB, C);
53      and U_AND03(a3, A, B, E) ;
54      and U_AND04(a4, nA, nB, nC, nD, E) ;
55      and U_AND05(a5, nA, B, C, D, nE) ;
56      or U_OR0(Da, a0, a1, a2, a3, a4, a5) ;
57
58      wire b0, b1, b2, b3, b4 ;
59      and U_AND06(b0, B, nD, nE);
60      and U_AND07(b1, B, nC, D) ;
61      and U_AND08(b2, B, C, E) ;
62      and U_AND09(b3, nB, nC, nD, E) ;
63      and U_AND10(b4, nB, C, D, nE) ;
64      or U_OR1(Db, b0, b1, b2, b3, b4) ;
65
66      wire c0, c1, c2, c3 ;
67      and U_AND11(c0, nC, nD, E) ;
68      and U_AND12(c1, nC, D, nE) ;
69      and U_AND13(c2, C, nD, nE) ;
70      and U_AND14(c3, C, D, E) ;
71      or U_OR2(Dc, c0, c1, c2, c3) ;
72
73      buf U_BUF(Dd, nD) ;
74
75
76  endmodule
77
```

Código en verilog

XII. Conclusión

Durante la realización de esta práctica se logró aplicar de manera integral los conocimientos adquiridos en la materia de lógica digital, incluyendo tanto circuitos combinacionales como secuenciales. Se trabajó con multiplexores, registros, flip-flops, contadores y conversores analógico-digitales y digital-analógicos, permitiendo comprender cómo se interrelacionan los diferentes bloques en un sistema digital completo.

El desarrollo de ejercicios prácticos, desde la creación de tablas de verdad y mapas de Karnaugh hasta la implementación de diagramas lógicos y códigos en Verilog, permitió reforzar la comprensión de conceptos teóricos y su traducción a aplicaciones reales. Además, se evidenció la importancia de optimizar funciones lógicas y diseñar circuitos robustos, como en los casos de FSM, contadores Johnson y anular, así como registros universales.

En resumen, la práctica consolidó la capacidad de diseñar, simular y programar sistemas digitales complejos, integrando análisis teórico, diseño lógico y codificación HDL, habilidades esenciales para el desarrollo de proyectos en electrónica y sistemas embebidos. También permitió apreciar la utilidad de herramientas modernas como Multisim y Verilog, que facilitan la validación y verificación de los diseños antes de su implementación física

XIII. Referencias

https://www.allaboutcircuits.com/

Calculadora Mapa de Karnaugh online

Sistema secuencial - Wikipedia, la enciclopedia libre

REGISyCONT8.PDF

Tema 1.1. Registros - Sistemas digitales II - Instituto Consorcio Clavijero